

1. Введение

Данный документ призван дать необходимые пояснения и советы для успешного выполнения НИРС по дисциплине «Цифровая обработка сигналов». В конечном итоге, при правильной реализации проекта, должен получиться готовый программный продукт, выполняющий функции фильтрации и модификации звука. Также здесь описаны рекомендации по применению паттернов (шаблонов) ООП (объектно-ориентированного программирования) для реализации масштабируемой и легко модифицируемой системы.

2. Общая структура проекта

Так как цель работы – это создание программы эквалайзера, то первоначально возникает вопрос получения звука для обработки. Один из самых простых и удобных способов - использование аудиофайлов. Довольно простым форматом для чтения является WAV файл, но не обязательно работать с ним вручную. Существует множество библиотек, которые предоставляют возможности для открытия всевозможных мультимедийных файлов и загрузки отсчетов, из которых они состоят, в память компьютера. Также для проигрывания медиа разрешается использовать средства, предоставляемые используемым в проекте языком. Поэтому, задача НИРС состоит в том, чтобы изменять звуковые отсчеты для реализации фильтрации и применения эффектов. Общая структура подобной программы показана на рис. 1.

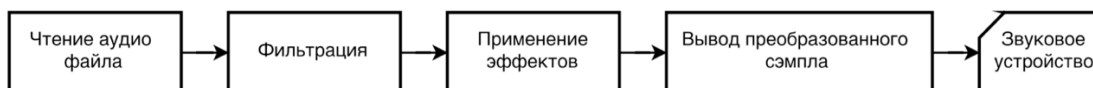


Рис. 1. Общая структура проекта

Как видно из схемы, описанные выше решения по проигрыванию медиафайлов, уже реализованные в различных библиотеках, выполняют чтение аудиофайла и вывод отсчета на звуковое устройство. Остается определиться с форматом данных, которые будут передаваться между описанными блоками. При чтении файла программисту, разумеется, доступна сразу вся аудиозапись для обработки. Но, учитывая такую возможность получения сигнала, как его запись, в реальном масштабе времени с микрофона, следует обрабатывать небольшое количество отсчетов за один раз. Соответственно, рациональное решение будет состоять в передаче нескольких или даже одного отсчета между программными модулями. Это позволит легко использовать систему для задач реального времени, позволяя избежать задержек при проигрывании звука, так как все отсчеты будут обрабатываться по мере надобности их воспроизведения. Следовательно, модули фильтрации отсчетов и применения эффектов следует проектировать таким образом, чтобы они принимали и возвращали небольшое число звуковых отсчетов за один раз.

3. Проектирование фильтра

Реализация продуктов практически в любой отрасли начинается с моделирования. Такой подход позволяет избежать множества ошибок и перепробовать разные методики разработки, лишней раз проверить достоинства и недостатки предполагаемых для использования моделей. То же самое касается и фильтров. Прежде чем начать писать программный код, следует понять, какой фильтр будет применен для поставленных целей. Помощником в этом является программный пакет Matlab, в частности, программа fdatools. Но, прежде чем приступить непосредственно к моделированию фильтров, следует определиться с частотными диапазонами, в которых они будут работать. Для их нахождения необходимо знать рабочие частоты эквалайзера и число полос, которые в нем можно регулировать. Рассмотрим пример расчета диапазонов фильтра. Пусть эквалайзер работает в частотах от 20 до 22000 герц (эти значения взяты потому, что звук с такими параметрами воспринимается человеческим ухом) и имеет 7 полос. Диапазон каждой из них должен быть в 2 раза шире предыдущей. Таким образом, мы сможем узнать сначала рабочие значения первой полосы, а остальные будут напрямую зависеть от ее значений. Из указанного выше условия получаем, что ее диапазон в 127 раз ($1 + 2 + 4 + 8 + 16 + 32 + 64$) меньше всего диапазона эквалайзера и, соответственно, составляет 173 герца ($21980 / 127$). Таким образом, можно получить длины всех полос. Их значения составляют (нумерация соответствует номеру полосы):

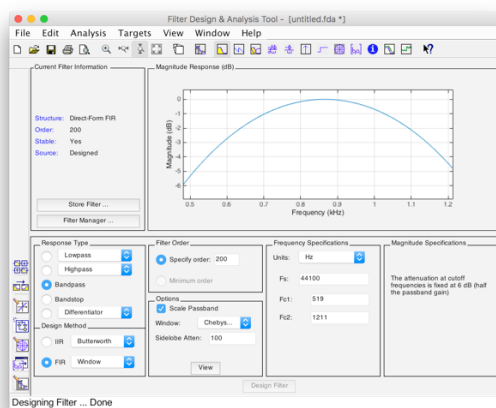
1. 173
2. 346
3. 692
4. 1384
5. 2768
6. 5536
7. 11072

Теперь можно составить сами диапазоны частот:

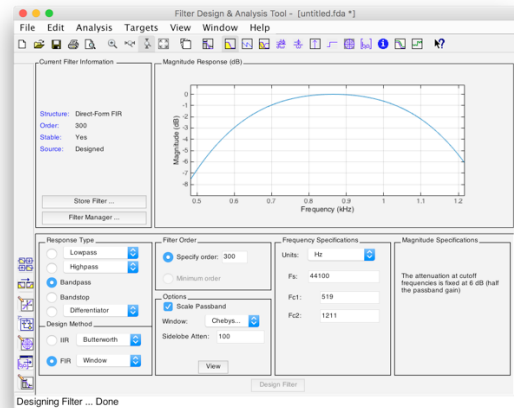
1. 0 – 173
2. 173 - 519
3. 519 – 1211
4. 1211 – 2595
5. 2595 – 5363
6. 5363 – 10899
7. 10899 – 21971

Зная рабочие частоты каждой из полос, можно приступить к непосредственному моделированию фильтров. Для этого потребуется пакет Matlab. После открытия данной программы нужно ввести в появившейся командной строке fdatool. Данная команда откроет встроенную в приложение утилиту для проектирования фильтров. Приведем пример проектирования КИХ (Конечная Импульсная Характеристика, на английском языке FIR) фильтра. Для того, чтобы фильтрация проходила в четко заданном диапазоне,

следует брать порядок фильтра не ниже 100-го. Во избежание проблем со щелчками, не рекомендуется делать спад магнитуды после граничных частот слишком резким, а дать ему продлиться примерно до 10% от диапазона полосы. Таким образом, получится перекрытие характеристик фильтров в эквалайзере. Путем построения характеристик разных моделей фильтров было установлено, что под указанные требования подходит оконный фильтр Чебышева. Он дает требуемый спад характеристики и возможность стабильного ослабления сигнала на 100 децибел (за пределами фронтов спада и нарастания характеристики). Частоту дискретизации каждого из фильтров следует брать равной наиболее распространенной в цифровом звуке, а именно 44100 герц. Пример подбора параметров для третьей полосы представлен на рис. 2 (хорошо видна характеристика у граничных значений работы фильтра) и рис. 3 (видна общая картина спада характеристики).

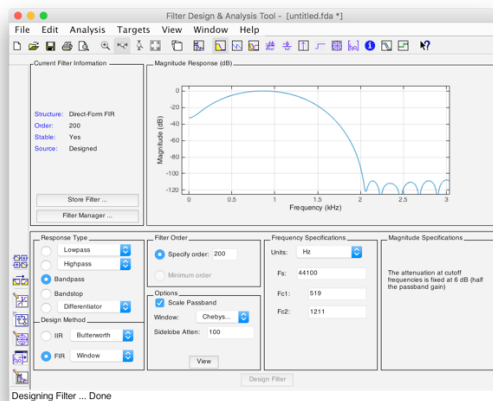


а)

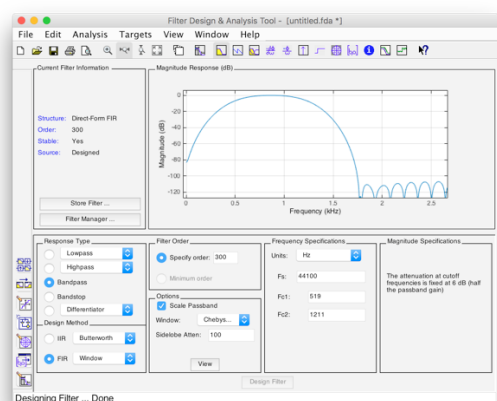


б)

Рис. 2. Параметры третьей полосы эквалайзера
а) Для порядка фильтра 200 б) Для порядка фильтра 300



а)



б)

Рис. 3. Параметры третьей полосы эквалайзера
а) Для порядка фильтра 200 б) Для порядка фильтра 300

Как видно из рис. 2, на спаде АЧХ у граничных значений работы фильтров нет большой разницы в выбранном порядке. Увеличив его на 100,

получили прирост всего в 1 децибел. Но если посмотреть на рис. 3, то можно увидеть, что общая картина фронтов характеристики при изменении порядка отличается уже намного серьезнее. Например, при порядке 200 на значении в 0 герц можно было добиться лишь ослабления сигнала на 40 децибел, а при порядке 300 – уже на 80 децибел. Порядок больше 300 выбирать не имеет смысла, так как полученный результат уже более чем удовлетворяет требованиям.

После того, как фильтр спроектирован, нужно получить его коэффициенты для использования в программе. Для этого следует в верхней панели выбрать меню Targets → Generate C header ... В следующем меню выбрать Export suggested (Double precision floating point). В результате будет создан header файл с коэффициентами фильтра. В нем необходимо изменить типы данных на double (так как Matlab предлагает для использования свои особые типы, добавлять которые нет смысла). Также, для удобства следует объединить все полученные файлы в один, переименовав массивы с коэффициентами во избежание коллизии имен.

4. Алгоритм фильтрации

Рассмотрим схему работы КИХ фильтра, представленную на рис. 4.

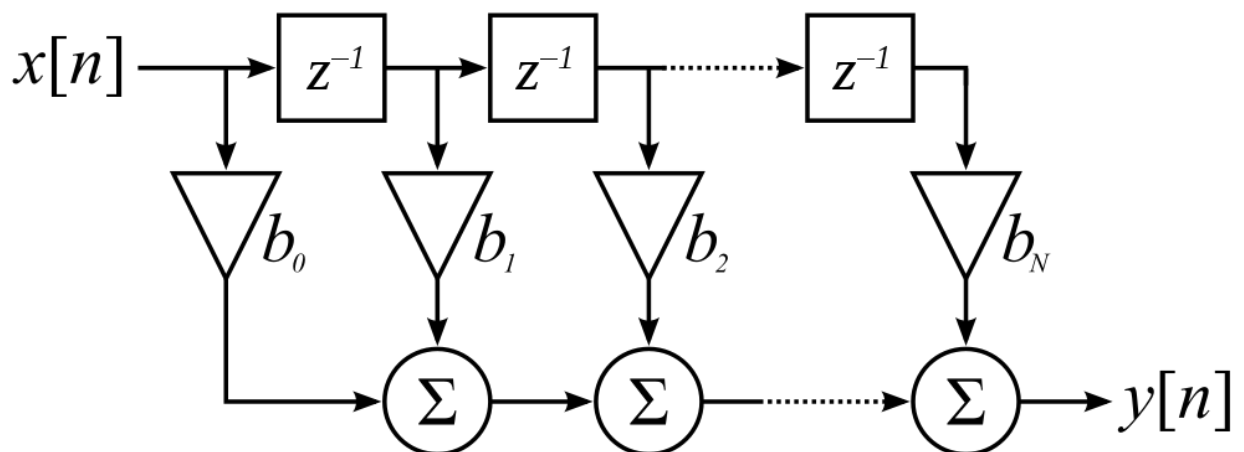


Рис. 4. Схема реализации прямой формы КИХ-фильтра

Здесь:

- $x[n]$ – это входной отсчет (текущий);
- Z^{-1} – предыдущий отсчет от текущего (соответственно, когда входной сигнал попадает в несколько таких элементов, то это будет предпредыдущий отсчет и так далее);
- b_N – один из отсчетов фильтра. Число N будет на 1 больше, чем порядок фильтра. Как следует из рис. 4, в памяти необходимо хранить лишь $N-1$ предыдущих отсчетов для корректной реализации фильтрации (так как нулевой – это текущий отсчет);

- Σ – сумматор результата. Все эти элементы складывают отсчеты, перемноженные с коэффициентами фильтра, и таким образом формируют выходной отсчет;
- $y[n]$ – выходной отсчет (результат фильтрации).

Таким образом, для получения нового отсчета на основе поступающего в данный момент следует:

1. Хранить буфер N предыдущих отсчетов (где N – порядок фильтра).
2. Перемножить коэффициенты фильтра с текущим входным отсчетом и с N предыдущими. Соответственно, одному коэффициенту будет соответствовать один отсчет.
3. Просуммировать $N + 1$ результатов вычислений, полученных на втором шаге.

Вычисляя по одному отсчету за такт, как это описано выше, можно избежать проблемы «стыков» фильтрации, которые возникают при обработке сразу группы отсчетов. Этот недочет возникает из-за того, что не учитываются предыдущие отсчеты в расчетах. В описанном выше методе фильтрации проблема отсутствует. Описанный алгоритм фильтрует лишь одну полосу звукового диапазона. Так как эквалайзер – это набор фильтров, то каждый отсчет следует отфильтровать всеми доступными полосами, а потом сложить все полученные значения. Результат как раз и является новым отсчетом, который пойдет на обработку блоком эффектов, а затем и на вывод звукового устройства.

5. Кольцевой буфер (очередь)

В предыдущем разделе шла речь о том, что существует необходимость записывать предыдущие отсчеты в буфер. Обычная его линейная структура не подойдет по той простой причине, что постоянно следует добавлять новый отсчет и стирать старый. Соответственно, данные в обычном буфере придется постоянно сдвигать на байт (такая возможность есть в языке Java по умолчанию). Однако, такое решение далеко не самое оптимальное. Значит, следует использовать такую структуру, у которой будет меняться начало буфера, а не смещаться данные. Принцип работы такого буфера представлен на рис. 5.

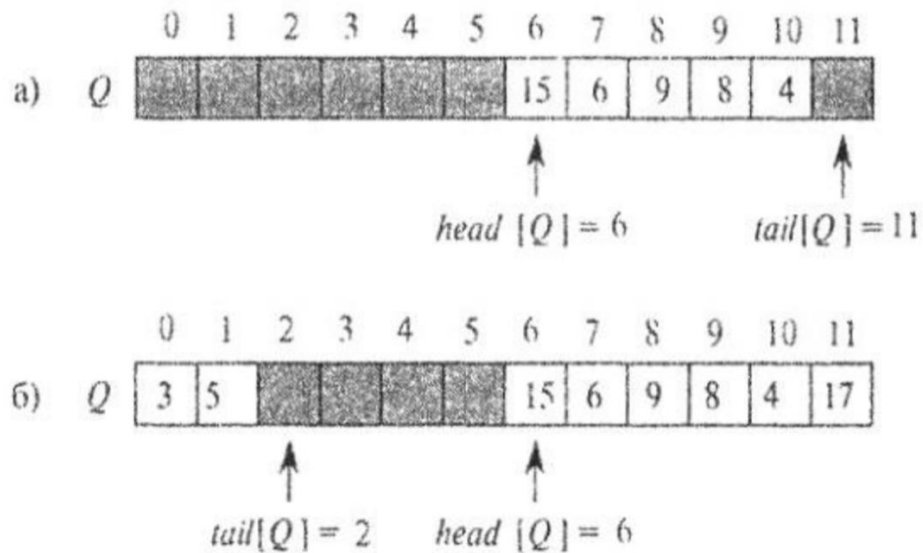


Рис. 5. Принцип работы кольцевого буфера

В состоянии а) начало буфера находится на позиции 6 одномерного массива, а хвост – на 11-ом элементе (последнем). Далее, нам нужно добавить в буфер числа 17, 3 и 5. С числом 17 все очевидно – оно запишется в свободную 11-ю ячейку. А вот для чисел 3 и 5 места уже нет. Но поскольку мы можем перезаписывать старые отсчеты, то запишем число 3 в начало массива, т.е. в ячейку 0, а число 5 – в ячейку 1. Так мы приходим к состоянию б), указанному на рис.5. Ниже приведена простая реализация такого буфера на языке C++.

```
// Очередь целых чисел, реализованная с помощью массива.
class CQueue {
public:
    explicit CQueue( int size );
    ~CQueue() { delete[] buffer; }
    // Добавление и извлечение элемента из очереди.
    void Enqueue( int a );
    int Dequeue();
    // Проверка на пустоту.
    bool IsEmpty() const { return head == tail; }
private:
    int* buffer;
    int bufferSize;
    int head; // Указывает на первый элемент очереди.
    int tail; // Указывает на следующий после последнего.
};

CQueue::CQueue( int size ) :
    bufferSize( size ),
    head( 0 ),
    tail( 0 )
{
    buffer = new int[bufferSize]; // Создаем буфер.
```

```

}
// Добавление элемента.
void CQueue::Enqueue( int a )
{
    assert ((tail + 1) % bufferSize!=head);
    buffer[tail] = a;
    tail = (tail + 1) % bufferSize;
}
// Извлечение элемента.
int CQueue::Dequeue()
{
    assert (head!=tail);
    int result = buffer[head];
    head = (head + 1) % bufferSize;
    return result;
}

```

Таким образом, возможно использование одномерного массива фиксированного размера с возможностью добавления новых отсчетов и удаления старых.

6. Организация многопоточной фильтрации сигнала

Учитывая тот факт, что подавляющее большинство современных вычислительных машин имеет двух- и более ядерные процессоры в своем составе, то возможно значительное сокращение времени, требуемого для фильтрации сигнала, путем организации параллельного вычисления значений отфильтрованных отсчетов для каждой полосы. На рис.6 приведена простая схема многопоточной обработки сигнала, построенная на основе паттерна (шаблона программирования) «обозреватель». Данный метод проектирования заключается в том, что в программе имеется сущность (в данном случае объект фильтра), которая может принимать сообщения от других сущностей и определенным образом реагировать на них.

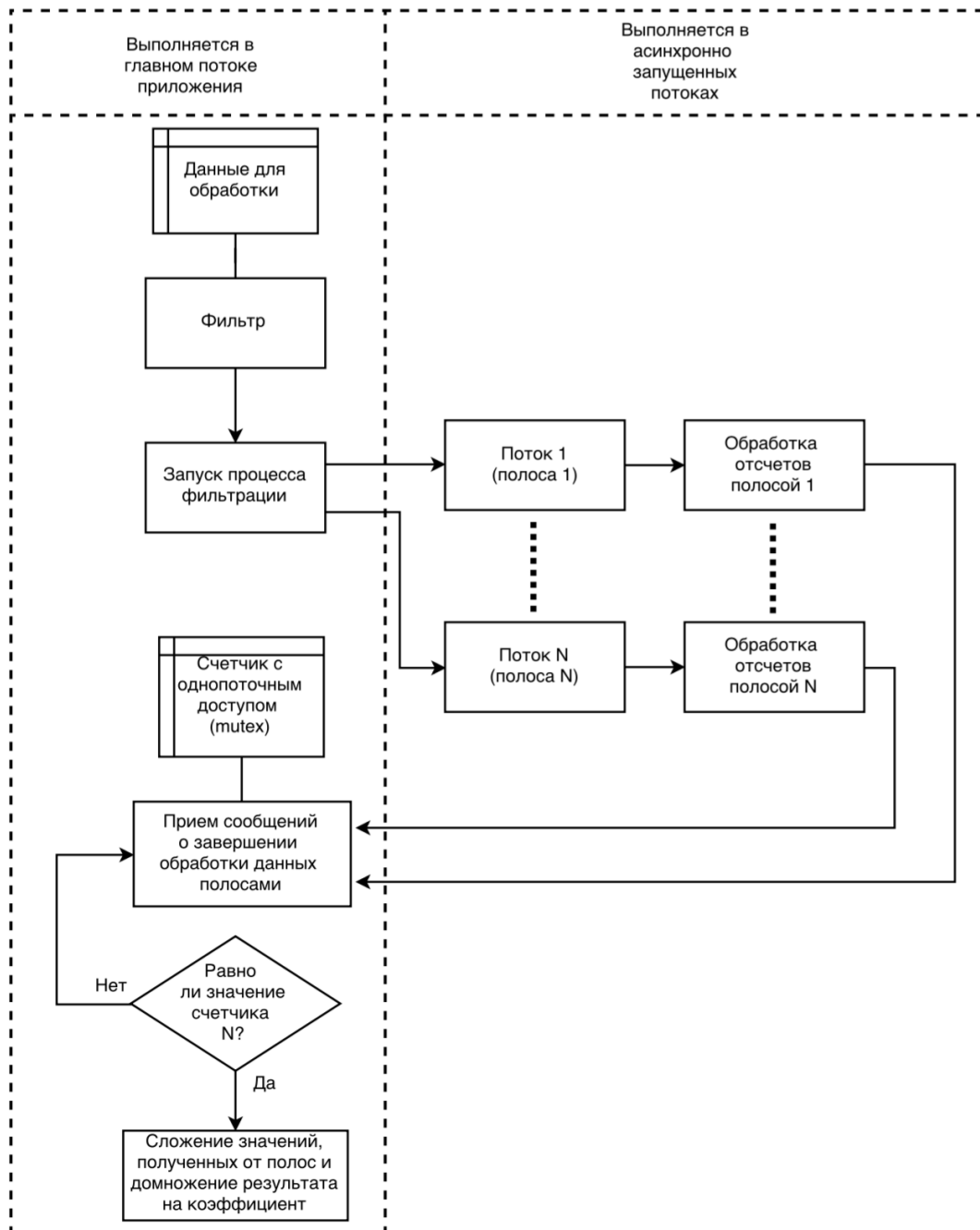


Рис. 6. Схема многопоточной фильтрации отсчетов

Рассмотрим алгоритм, изображенный на рис.6 более подробно. Его работа начинается в тот момент, когда фильтру поступают новые данные для обработки. При этом вызывается метод, запускающий процесс фильтрации. Этот метод вызывает асинхронно (обособлено от главного потока) методы фильтрации у каждой из полос фильтра. Соответственно, обработка отсчетов каждой полосой идет независимо друг от друга. Таким образом, возникает необходимость в установлении момента, когда будет завершена обработка данных всеми потоками. Для этого как раз и следует реализовать механизм приема сообщений объектом фильтра. Он заключается в следующем. По завершении процесса обработки полосой вызывается метод объекта фильтра, прибавляющий к специальному счетчику единицу. Так ведется счет потоков,

которые завершили свою работу. Особенность этого счетчика в том, что к нему должен предоставляться только однопоточный доступ. Это возможно реализовать используя, например, mutex или семафор в языках C / C++. В методе, в котором происходит инкрементация (увеличение на единицу) значения счетчика, после произведенного действия следует проверить, не равно ли это значение числу полос фильтра (N). Если да, то это означает, что все асинхронные процессы завершили свою работу и теперь полученные от этих процессов значения можно сложить и передать на дальнейшую обработку блоку эффектов. В некоторых языках программирования потоки реализованы таким образом, что переключение между ними является довольно затратной задачей. Потребность в переключении возникает тогда, когда число потоков превышает число установленных в компьютере физических вычислительных ядер. Следовательно, в таком случае следует объединять вычисление нескольких полос в один поток. Данная оптимизация будет работать для таких языков как Java или C / C++. Но для таких языков, как Erlang или Go она не будет иметь смысла, так как в них реализация механизма переключения между потоками принципиально другая и не отнимает большого числа ресурсов машины.

7. Пример организации многопоточной обработки данных

Сначала рассмотрим главный класс, который иницирует создание всех нужных объектов и обработку данных – Equalizer.

```
public class Equalizer {  
  
    public static void main(String[] args) {  
        //Создаем объект фильтра  
        Filter filter = new Filter();  
  
        double coefs1[] = {0.1 ,1.1};  
        double coefs2[] = {0.2 ,1.2};  
        double coefs3[] = {0.3 ,1.3};  
  
        /*  
        Создаем объекты полос, привязываем их  
        к объекту фильтра для получения  
        сообщений о готовности  
        */  
        new Band(filter, "Band 1", coefs1);  
        new Band(filter, "Band 2", coefs2);  
        new Band(filter, "Band 3", coefs3);  
  
        double data[] = {0.0 ,1.1, 2.2, 3.3};  
        //Передача данных фильтру для обработки  
        filter.setData(data);  
    }  
}
```

Далее, рассмотрим типовую структуру объекта фильтра (Filter), который получает сообщения от объектов полос (Band).

```

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class Filter {
    //Счетчик числа полос, завершивших работу
    private AtomicInteger completed = new AtomicInteger(0);
    //Массив указателей на объекты полос
    private List<Observer> bands = new ArrayList<>();
    private double[] data;

    public void notifyCompleted() {
        //Повышаем значение счетчика
        completed.incrementAndGet();
        System.out.println("'" + bands.size());
        if (bands.size() == completed.get()) {
            /*
             * Все полосы завершили работу, здесь
             * следует произвести сложение результатов
             * работы полос
             */
            System.out.println("All tasks were finished");
        }
    }

    public double[] getData() {
        return data;
    }

    /*
     * Как только фильтр получает новые данные,
     * он должен дать их на обработку всем
     * полосам
     */
    public void setData(double[] data) {
        this.data = data;
        notifyAllObservers();
    }

    public void attach(Observer observer) {
        bands.add(observer);
    }

    //Запуск обработки данных полосами
    public void notifyAllObservers() {
        completed.set(0);
        for (Observer observer : bands) {
            observer.update();
        }
    }
}

```

Для объекта полосы можно создать простенький интерфейс, он будет выглядеть так:

```

//Интерфейс объекта полосы
public abstract class Observer {
    protected Filter filter;
    public abstract void update();
}

```

Далее представлен класс, реализующий вышеописанный интерфейс:

```
public class Band extends Observer {

    protected Filter filter;
    private double[] coefs;
    private String name;

    public Band(Filter filter, String name, double[] coefs) {
        this.filter = filter;
        this.filter.attach(this);
        this.coefs = coefs;
        this.name = name;
    }

    @Override
    public void update() {
        //Работа полосы фильтра, происходит в асинхронном потоке
        new Thread(new Runnable() {
            @Override
            public void run() {
                double[] data = filter.getData();
                System.out.println(name + " started.");
                System.out.print("{");
                for (double coef : coefs) {
                    for (double sample : data) {
                        System.out.print(coef * sample + ",");
                    }
                }
                System.out.print("}\n");
                /*
                 Оповещение о завершении работы полосы
                 (здесь можно передать фильтру массив
                 обработанных данных)
                 */
                filter.notifyCompleted();
            }
        }).run();
    }
}
```

8. Реализация эффектов

В данной части методического указания кратко описываются принципы обработки сигнала для достижения различных звуковых эффектов. Рассмотрим по порядку реализацию следующих эффектов:

1. Delay и Echo. Данный эффект осуществляет наложение на текущий звуковой отсчет еще один, прошедший заданное время назад с некоторым коэффициентом затухания. Есть также очень похожий на него эффект Echo. Их различие состоит в том, что при использовании эффекта Delay старые отсчеты будут многократно накладываться на новые, пока коэффициент затухания не приблизит их к нулю в процессе многократного повторения, а при использовании Echo старый отсчет может добавиться к новому

лишь один раз. Для реализации обоих эффектов нужно хранить кольцевой буфер размером в такое количество отсчетов, на которое должна происходить задержка сигнала. При старте программы все значения буфера должны быть проинициализированы нулями. При обработке каждого отсчета нужно брать самый последний отсчет из буфера, умножать его на коэффициент затухания и прибавлять полученное значение к значению текущего отсчета. При реализации эффекта Delay полученный результат следует записать в кольцевой буфер, а при реализации Echo в буфер следует записать отсчет до обработки эффектом. Таким образом, старый отсчет, который добавлялся в этом цикле к сигналу, будет стерт, а новый записан в начало очереди с каждым новым отсчетом приближаясь к ее концу, и, как к следствие, к моменту его использования. Оба эффекта следует использовать в конце цепочки эффектов (то есть непосредственно перед выводом на аудиоустройство). Регулируемый параметр - время задержки, которое очень просто изменять в данной реализации. Следует определиться с максимальным ее значением, ибо относительно него и задается размер кольцевого буфера. Путем уменьшения переменной `bufferSize` (см. пример реализации кольцевого буфера) можно добиться уменьшения времени задержки.

2. Chorus. При реализации данного эффекта используется параллельное включение нескольких блоков эффекта Delay с небольшой временной задержкой сигнала, которая меняется для каждого вычисляемого отсчета случайным образом. По окончании обработки ими сигнала результат складывается.
3. Reverb. Тот же эффект, что и Echo, но время задержки выставляется крайне низким (около 50 миллисекунд). Такая короткая задержка не воспринимается мозгом как отдельный сигнал, а потому слышится как несколько другой звуковой эффект, нежели Echo (в котором ясно слышится наложенный поверх текущего задержанный сигнал).
4. Vibrato. Данный эффект создает дребезжание обрабатываемого звука. Реализуется путем перемножения звукового сигнала с синусоидой, чье значение амплитуды должно периодически меняться от 1 до -1. Регулируемый параметр для этого эффекта – длина периода синусоиды в отсчетах, но важно иметь в виду, что он не должен быть слишком большим, иначе получится уже другой эффект.
5. Overdrive и Distortion. Два очень похожих как по звучанию, так и по реализации эффекта. Оба характеризуются «рваным» звуком. Их реализация состоит в отсечении амплитуды сигнала выше некоторого порога. При реализации эффекта Distortion амплитуда отсекается выше определенного значения, без сглаживания. Для того, чтобы реализовать эффект Overdrive требуется сгладить участки до и после отсечения амплитуды. В обеих реализациях

следует учитывать то, что амплитуда может иметь отрицательное значение.

9. Обработка переполнения

Возможна ситуация, когда полученное для вывода на звуковое устройство значение отсчета будет выше разрядности отсчетов в звуковых файлах (которая равняется 16 битам, иначе это тип `int16_t` в языке C / C++). Это может произойти как в процессе фильтрации, так и в процессе обработки сигнала блоком эффектов. Соответственно, перед последующим выводом звукового отсчета следует убедиться, что в нем отсутствует переполнение. Для этого нужно проверить, не превышает ли его значение максимально возможное для типа `int16_t` (в языках C / C++ предопределенная (`#define`) константа `INT16_MAX`). Если значение превышает, то следует вычислить коэффициент, на который будут умножаться все последующие отсчеты, чтобы держать одинаковый уровень звука в дальнейшем. Вычисляется он путем деления константы `INT16_MAX` на значение переполненного отсчета.